

09_week9_gradient_boosting

July 30, 2026

1 Week 9: Gradient Boosting on My Two Datasets

For this week, I wanted to try gradient boosting on both of the supervised datasets I have been using in this project:

1. Wisconsin breast cancer diagnosis
2. Brazil patient outcome dataset

The main thing I wanted to see was how gradient boosting compares to random forest on each one. I also wanted to keep this notebook pretty straightforward, so I am doing the model fitting and graphs directly in the notebook instead of hiding a lot of the work in helper functions.

```
[17]: from pathlib import Path
import sys

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.ensemble import RandomForestClassifier,
    GradientBoostingClassifier, HistGradientBoostingClassifier
from sklearn.inspection import PartialDependenceDisplay, permutation_importance
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_recall_curve,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import GridSearchCV, StratifiedKFold,
    train_test_split

project_root = Path.cwd().resolve()
if not (project_root / "src").exists():
```

```

    project_root = project_root.parent
if str(project_root) not in sys.path:
    sys.path.append(str(project_root))

from src.load_data import load_wisconsin, load_brazil, split_X_y

sns.set_theme(style="whitegrid", context="notebook")
pd.set_option("display.max_columns", 100)

```

1.1 1. Wisconsin Dataset

I started with Wisconsin because it is smaller and cleaner, which makes it a good dataset for testing the full gradient boosting workflow.

In this section, I am checking the class balance, splitting the data into training and test sets, tuning a few gradient boosting settings, and then comparing the results to random forest.

```

[18]: wisconsin_df, wisconsin_target = load_wisconsin()
X_w, y_w = split_X_y(wisconsin_df, wisconsin_target)

print("Wisconsin shape:", wisconsin_df.shape)
display(y_w.value_counts().rename("count").to_frame())
display((y_w.value_counts(normalize=True).rename("share") * 100).round(2).
        ↪to_frame())

X_w_train, X_w_test, y_w_train, y_w_test = train_test_split(
    X_w, y_w, test_size=0.25, random_state=42, stratify=y_w
)

print("Training rows:", X_w_train.shape[0])
print("Test rows:", X_w_test.shape[0])

```

Wisconsin shape: (569, 31)

	count
Diagnosis	
0	357
1	212

	share
Diagnosis	
0	62.74
1	37.26

Training rows: 426

Test rows: 143

```

[19]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

wisconsin_rf = RandomForestClassifier(

```

```

        n_estimators=300,
        min_samples_leaf=2,
        random_state=42,
        n_jobs=-1,
    )
    wisconsin_rf.fit(X_w_train, y_w_train)

    wisconsin_gb_grid = {
        "n_estimators": [100, 200, 300],
        "learning_rate": [0.03, 0.05, 0.1],
        "max_depth": [1, 2, 3],
        "subsample": [0.8, 1.0],
    }

    wisconsin_gb_search = GridSearchCV(
        estimator=GradientBoostingClassifier(random_state=42),
        param_grid=wisconsin_gb_grid,
        scoring="roc_auc",
        cv=cv,
        n_jobs=-1,
    )
    wisconsin_gb_search.fit(X_w_train, y_w_train)

    wisconsin_gb = wisconsin_gb_search.best_estimator_
    print("Best Wisconsin gradient boosting parameters:")
    print(wisconsin_gb_search.best_params_)

```

Best Wisconsin gradient boosting parameters:

```
{'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 300, 'subsample': 0.8}
```

```

[20]: wisconsin_results = []

for model_name, model in {
    "Random Forest": wisconsin_rf,
    "Gradient Boosting": wisconsin_gb,
}.items():
    y_pred = model.predict(X_w_test)
    y_score = model.predict_proba(X_w_test)[: , 1]

    wisconsin_results.append(
        {
            "model": model_name,
            "accuracy": accuracy_score(y_w_test, y_pred),
            "balanced_accuracy": balanced_accuracy_score(y_w_test, y_pred),
            "f1": f1_score(y_w_test, y_pred),
            "roc_auc": roc_auc_score(y_w_test, y_score),
            "average_precision": average_precision_score(y_w_test, y_score),

```

```

    }
)

wisconsin_results_df = pd.DataFrame(wisconsin_results).sort_values("roc_auc",
↪ascending=False)
display(wisconsin_results_df.round(4))

```

	model	accuracy	balanced_accuracy	f1	roc_auc \
1	Gradient Boosting	0.965	0.9528	0.9505	0.9977
0	Random Forest	0.965	0.9528	0.9505	0.9958

	average_precision
1	0.9966
0	0.9935

```

[21]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for model_name, model, color in [
    ("Random Forest", wisconsin_rf, "#3b82f6"),
    ("Gradient Boosting", wisconsin_gb, "#ef4444"),
]:
    y_pred = model.predict(X_w_test)
    y_score = model.predict_proba(X_w_test)[: , 1]

    fpr, tpr, _ = roc_curve(y_w_test, y_score)
    precision, recall, _ = precision_recall_curve(y_w_test, y_score)

    axes[0].plot(fpr, tpr, label=model_name, color=color, linewidth=2)
    axes[1].plot(recall, precision, label=model_name, color=color, linewidth=2)

cm = confusion_matrix(y_w_test, wisconsin_gb.predict(X_w_test))
sns.heatmap(cm, annot=True, fmt="d", cmap="Reds", cbar=False, ax=axes[2])

axes[0].plot([0, 1], [0, 1], linestyle="--", color="gray")
axes[0].set_title("Wisconsin ROC Curve")
axes[0].set_xlabel("False Positive Rate")
axes[0].set_ylabel("True Positive Rate")
axes[0].legend()

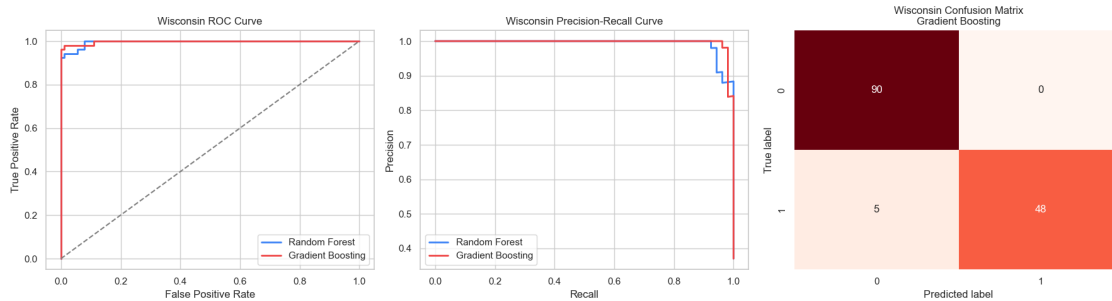
axes[1].set_title("Wisconsin Precision-Recall Curve")
axes[1].set_xlabel("Recall")
axes[1].set_ylabel("Precision")
axes[1].legend()

axes[2].set_title("Wisconsin Confusion Matrix\nGradient Boosting")
axes[2].set_xlabel("Predicted label")
axes[2].set_ylabel("True label")

```

```
plt.tight_layout()
plt.show()

print("Wisconsin classification report for gradient boosting:")
print(classification_report(y_w_test, wisconsin_gb.predict(X_w_test)))
```

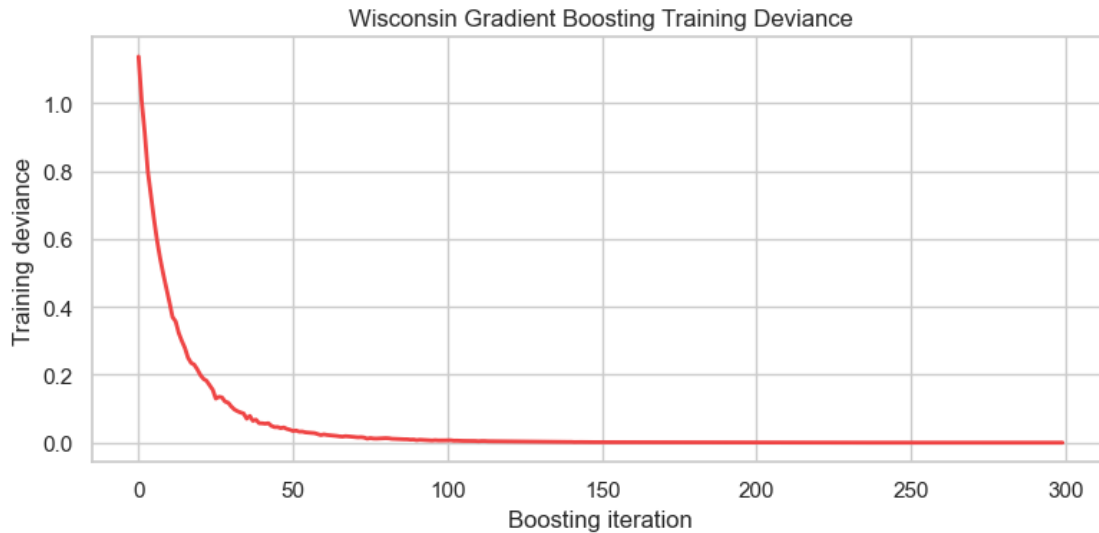


Wisconsin classification report for gradient boosting:

	precision	recall	f1-score	support
0	0.95	1.00	0.97	90
1	1.00	0.91	0.95	53
accuracy			0.97	143
macro avg	0.97	0.95	0.96	143
weighted avg	0.97	0.97	0.96	143

```
[22]: fig, ax = plt.subplots(figsize=(8, 4))

ax.plot(wisconsin_gb.train_score_, color="#ef4444", linewidth=2)
ax.set_title("Wisconsin Gradient Boosting Training Deviance")
ax.set_xlabel("Boosting iteration")
ax.set_ylabel("Training deviance")
plt.tight_layout()
plt.show()
```



```
[38]: wisconsin_perm = permutation_importance(
    wisconsin_gb,
    X_w_test,
    y_w_test,
    n_repeats=20,
    random_state=42,
    scoring="roc_auc",
)

wisconsin_importance = (
    pd.DataFrame(
        {
            "feature": X_w.columns,
            "importance": wisconsin_perm.importances_mean,
        }
    )
    .sort_values("importance", ascending=False)
    .head(10)
)

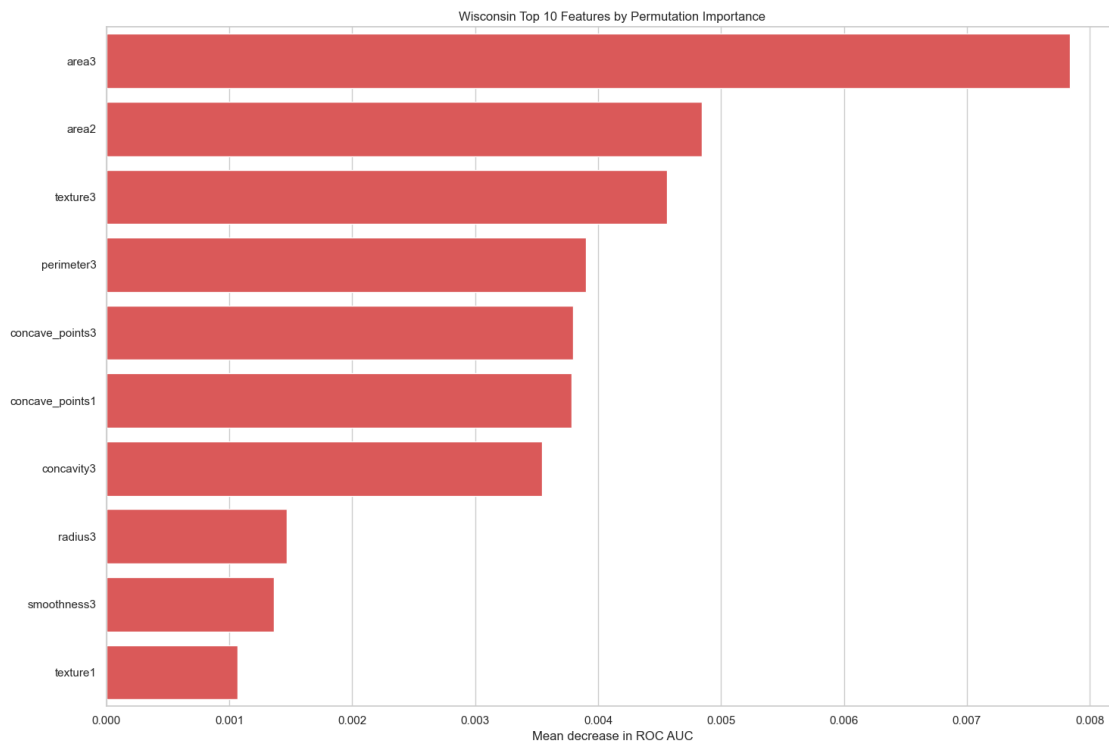
plt.figure(figsize=(15, 10))
sns.barplot(data=wisconsin_importance, x="importance", y="feature",
            color="#ef4444")
plt.title("Wisconsin Top 10 Features by Permutation Importance")
plt.xlabel("Mean decrease in ROC AUC")
plt.ylabel("")
plt.tight_layout()
plt.show()
```

```

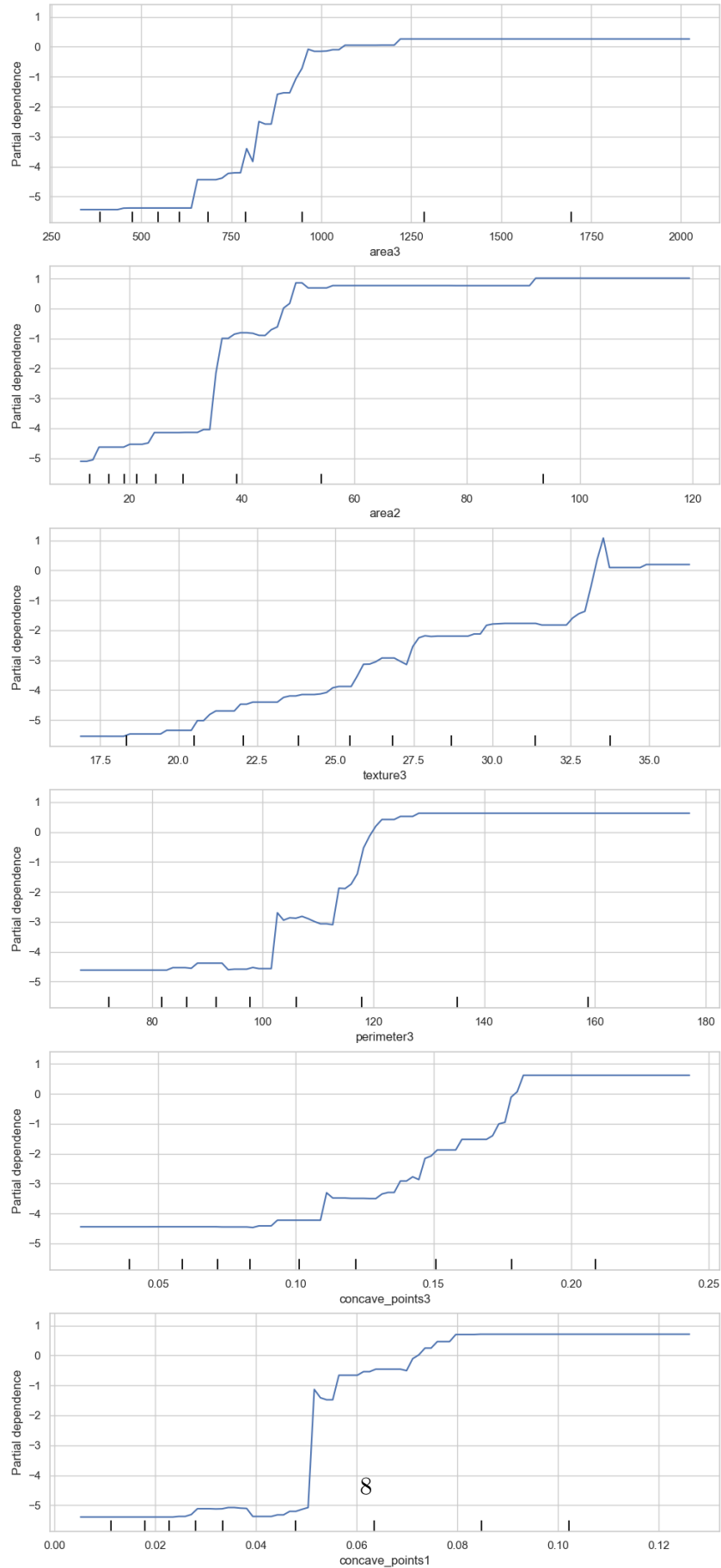
top_features = wisconsin_importance["feature"].head(6).tolist()

fig, ax = plt.subplots(figsize=(10, 22))
PartialDependenceDisplay.from_estimator(
    wisconsin_gb,
    X_w_train,
    features=top_features,
    n_cols=1,
    ax=ax,
)
plt.suptitle("Wisconsin Partial Dependence Plots for Top 6 Features", y=1.01)
plt.tight_layout()
plt.show()

```



Wisconsin Partial Dependence Plots for Top 6 Features



1.1.1 Wisconsin Takeaway

For Wisconsin, regular gradient boosting works well because the dataset is small enough that I can tune it without making the workflow too heavy.

The graphs that help the most here are the ROC curve, precision-recall curve, confusion matrix, training deviance over the boosting iterations, and the feature interpretation plots.

1.2 2. Brazil Dataset

The Brazil dataset needed a different approach because it is much larger and the target is more imbalanced.

Instead of copying the exact same setup from Wisconsin, I used a more practical workflow: sample a manageable number of rows, keep a random forest baseline, use HistGradientBoostingClassifier, and pay more attention to ROC AUC, average precision, and threshold tradeoffs.

```
[24]: brazil_sample_size = 120_000

brazil_df, brazil_target = load_brazil(sample_size=brazil_sample_size,
    ↪random_state=42, rename_cols=True)
X_b, y_b = split_X_y(brazil_df, brazil_target)

print("Brazil sample shape:", brazil_df.shape)
display(y_b.value_counts().rename("count").to_frame())
display((y_b.value_counts(normalize=True).rename("share") * 100).round(2).
    ↪to_frame())

X_b_train, X_b_test, y_b_train, y_b_test = train_test_split(
    X_b, y_b, test_size=0.25, random_state=42, stratify=y_b
)

print("Training rows:", X_b_train.shape[0])
print("Test rows:", X_b_test.shape[0])
```

Brazil sample shape: (120000, 40)

	count
deceased	
0	111602
1	8398

	share
deceased	
0	93.0
1	7.0

Training rows: 90000

Test rows: 30000

```
[25]: brazil_rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    min_samples_leaf=10,
    class_weight="balanced_subsample",
    random_state=42,
    n_jobs=-1,
)
brazil_rf.fit(X_b_train, y_b_train)

brazil_hgb_grid = {
    "learning_rate": [0.03, 0.05, 0.1],
    "max_depth": [3, 5, None],
    "max_leaf_nodes": [15, 31],
    "min_samples_leaf": [50, 200],
    "l2_regularization": [0.0, 1.0],
}

brazil_hgb_search = GridSearchCV(
    estimator=HistGradientBoostingClassifier(
        max_iter=250,
        early_stopping=True,
        validation_fraction=0.1,
        random_state=42,
    ),
    param_grid=brazil_hgb_grid,
    scoring="average_precision",
    cv=3,
    n_jobs=-1,
)
brazil_hgb_search.fit(X_b_train, y_b_train)

brazil_hgb = brazil_hgb_search.best_estimator_
print("Best Brazil HistGradientBoosting parameters:")
print(brazil_hgb_search.best_params_)
```

Best Brazil HistGradientBoosting parameters:

```
{'l2_regularization': 0.0, 'learning_rate': 0.05, 'max_depth': None,
 'max_leaf_nodes': 31, 'min_samples_leaf': 50}
```

```
[26]: brazil_results = []

for model_name, model in {
    "Random Forest": brazil_rf,
    "HistGradientBoosting": brazil_hgb,
```

```

}.items():
    y_pred = model.predict(X_b_test)
    y_score = model.predict_proba(X_b_test)[:, 1]

    brazil_results.append(
        {
            "model": model_name,
            "accuracy": accuracy_score(y_b_test, y_pred),
            "balanced_accuracy": balanced_accuracy_score(y_b_test, y_pred),
            "f1": f1_score(y_b_test, y_pred),
            "roc_auc": roc_auc_score(y_b_test, y_score),
            "average_precision": average_precision_score(y_b_test, y_score),
        }
    )

brazil_results_df = pd.DataFrame(brazil_results).
    ↪sort_values("average_precision", ascending=False)
display(brazil_results_df.round(4))

```

	model	accuracy	balanced_accuracy	f1	roc_auc \
1	HistGradientBoosting	0.9535	0.7350	0.5917	0.9584
0	Random Forest	0.8833	0.8793	0.5120	0.9508

	average_precision
1	0.6751
0	0.6392

```

[27]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for model_name, model, color in [
    ("Random Forest", brazil_rf, "#2563eb"),
    ("HistGradientBoosting", brazil_hgb, "#dc2626"),
]:
    y_score = model.predict_proba(X_b_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_b_test, y_score)
    precision, recall, _ = precision_recall_curve(y_b_test, y_score)

    axes[0].plot(fpr, tpr, label=model_name, color=color, linewidth=2)
    axes[1].plot(recall, precision, label=model_name, color=color, linewidth=2)

cm = confusion_matrix(y_b_test, brazil_hgb.predict(X_b_test))
sns.heatmap(cm, annot=True, fmt="d", cmap="Reds", cbar=False, ax=axes[2])

axes[0].plot([0, 1], [0, 1], linestyle="--", color="gray")
axes[0].set_title("Brazil ROC Curve")
axes[0].set_xlabel("False Positive Rate")
axes[0].set_ylabel("True Positive Rate")

```

```

axes[0].legend()

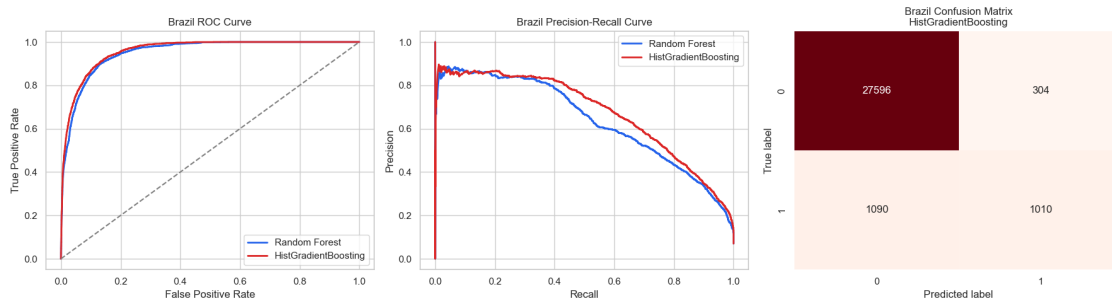
axes[1].set_title("Brazil Precision-Recall Curve")
axes[1].set_xlabel("Recall")
axes[1].set_ylabel("Precision")
axes[1].legend()

axes[2].set_title("Brazil Confusion Matrix\nHistGradientBoosting")
axes[2].set_xlabel("Predicted label")
axes[2].set_ylabel("True label")

plt.tight_layout()
plt.show()

print("Brazil classification report for HistGradientBoosting:")
print(classification_report(y_b_test, brazil_hgb.predict(X_b_test)))

```



```

Brazil classification report for HistGradientBoosting:
      precision    recall  f1-score   support

    0       0.96       0.99       0.98       27900
    1       0.77       0.48       0.59        2100

   accuracy                   0.95       30000
  macro avg       0.87       0.74       0.78       30000
 weighted avg       0.95       0.95       0.95       30000

```

```

[28]: brazil_scores = brazil_hgb.predict_proba(X_b_test)[: , 1]
      thresholds = np.linspace(0.05, 0.50, 19)

      threshold_rows = []
      for threshold in thresholds:
          threshold_pred = (brazil_scores >= threshold).astype(int)
          threshold_rows.append(
              {

```

```

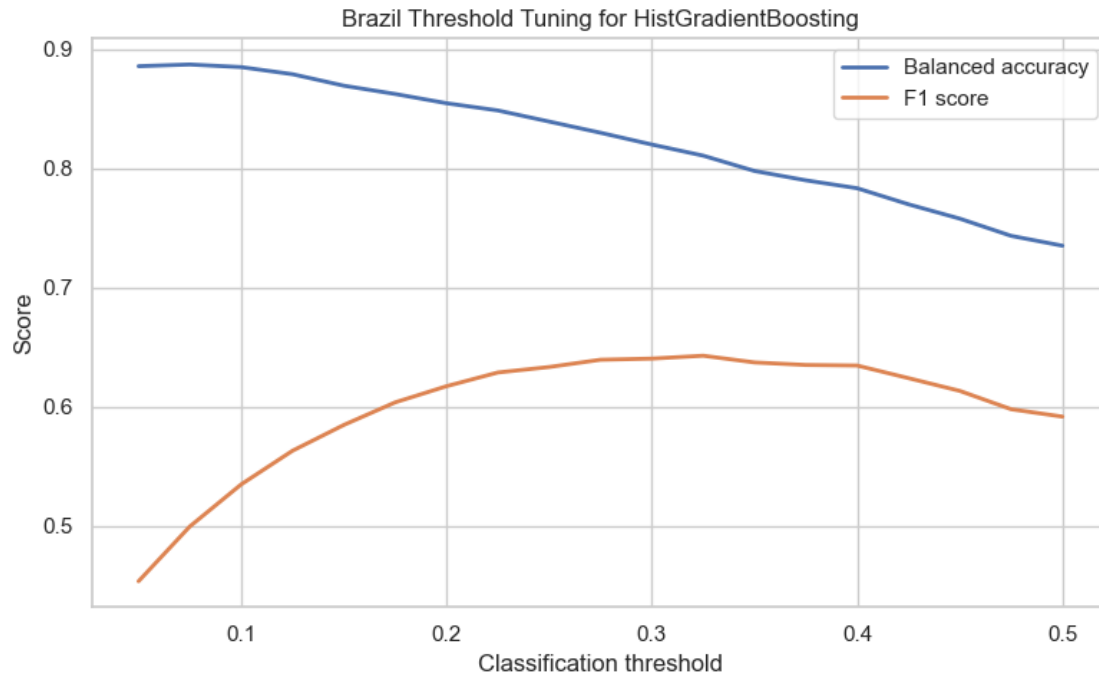
        "threshold": threshold,
        "balanced_accuracy": balanced_accuracy_score(y_b_test,
↪threshold_pred),
        "f1": f1_score(y_b_test, threshold_pred),
    }
)

threshold_df = pd.DataFrame(threshold_rows)
display(threshold_df.round(4))

plt.figure(figsize=(8, 5))
plt.plot(threshold_df["threshold"], threshold_df["balanced_accuracy"],
↪label="Balanced accuracy", linewidth=2)
plt.plot(threshold_df["threshold"], threshold_df["f1"], label="F1 score",
↪linewidth=2)
plt.title("Brazil Threshold Tuning for HistGradientBoosting")
plt.xlabel("Classification threshold")
plt.ylabel("Score")
plt.legend()
plt.tight_layout()
plt.show()

```

	threshold	balanced_accuracy	f1
0	0.050	0.8856	0.4536
1	0.075	0.8870	0.4996
2	0.100	0.8848	0.5349
3	0.125	0.8789	0.5631
4	0.150	0.8692	0.5847
5	0.175	0.8623	0.6037
6	0.200	0.8545	0.6172
7	0.225	0.8485	0.6287
8	0.250	0.8392	0.6333
9	0.275	0.8298	0.6394
10	0.300	0.8199	0.6403
11	0.325	0.8106	0.6428
12	0.350	0.7977	0.6371
13	0.375	0.7899	0.6350
14	0.400	0.7833	0.6345
15	0.425	0.7698	0.6240
16	0.450	0.7577	0.6133
17	0.475	0.7433	0.5979
18	0.500	0.7350	0.5917



```
[29]: brazil_sample_idx = X_b_test.sample(n=5000, random_state=42).index

brazil_perm = permutation_importance(
    brazil_hgb,
    X_b_test.loc[brazil_sample_idx],
    y_b_test.loc[brazil_sample_idx],
    n_repeats=10,
    random_state=42,
    scoring="average_precision",
)

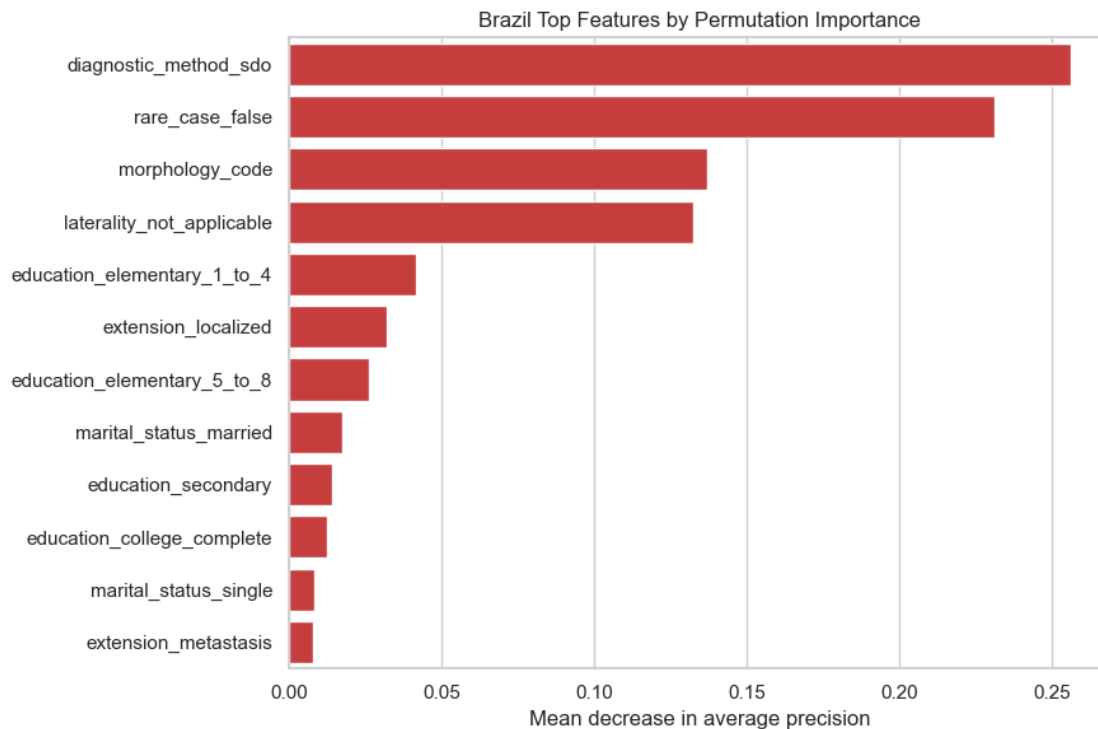
brazil_importance = (
    pd.DataFrame(
        {
            "feature": X_b.columns,
            "importance": brazil_perm.importances_mean,
        }
    )
    .sort_values("importance", ascending=False)
    .head(12)
)

plt.figure(figsize=(9, 6))
```

```

sns.barplot(data=brazil_importance, x="importance", y="feature",
            color="#dc2626")
plt.title("Brazil Top Features by Permutation Importance")
plt.xlabel("Mean decrease in average precision")
plt.ylabel("")
plt.tight_layout()
plt.show()

```



1.2.1 Brazil Takeaway

For Brazil, gradient boosting still makes sense, but I had to adjust the workflow to match the size and balance of the data.

The biggest points here are using the histogram-based version for speed, evaluating the model with class imbalance in mind, looking closely at the precision-recall curve, and checking whether a different classification threshold works better than 0.50.

```

[39]: # Report cross-validation performance for the already-selected models.
      # This refits only the winning parameter sets; it does not repeat either grid
      # search.
      from sklearn.model_selection import cross_validate

      gb_cv_scoring = {
          "roc_auc": "roc_auc",

```

```

        "average_precision": "average_precision",
    }

    wisconsin_gb_cv = cross_validate(
        wisconsin_gb,
        X_w_train,
        y_w_train,
        cv=cv,
        scoring=gb_cv_scoring,
        n_jobs=1,
    )

    brazil_gb_cv = cross_validate(
        brazil_hgb,
        X_b_train,
        y_b_train,
        cv=3,
        scoring=gb_cv_scoring,
        n_jobs=1,
    )

    gb_cv_results_df = pd.DataFrame(
        [
            {
                "dataset": "Wisconsin",
                "model": "Gradient Boosting",
                "cv_folds": 5,
                "cv_roc_auc_mean": wisconsin_gb_cv["test_roc_auc"].mean(),
                "cv_roc_auc_sd": wisconsin_gb_cv["test_roc_auc"].std(),
                "cv_pr_auc_mean": wisconsin_gb_cv["test_average_precision"].mean(),
                "cv_pr_auc_sd": wisconsin_gb_cv["test_average_precision"].std(),
            },
            {
                "dataset": "Brazil",
                "model": "HistGradientBoosting",
                "cv_folds": 3,
                "cv_roc_auc_mean": brazil_gb_cv["test_roc_auc"].mean(),
                "cv_roc_auc_sd": brazil_gb_cv["test_roc_auc"].std(),
                "cv_pr_auc_mean": brazil_gb_cv["test_average_precision"].mean(),
                "cv_pr_auc_sd": brazil_gb_cv["test_average_precision"].std(),
            },
        ]
    )

    print(gb_cv_results_df.round(4).to_string(index=False))

```

dataset	model	cv_folds	cv_roc_auc_mean	cv_roc_auc_sd
Wisconsin	Gradient Boosting	5	0.9929	0.0051

0.9909	0.0058			
Brazil	HistGradientBoosting	3	0.9577	0.0024
0.6803	0.0170			

1.3 Final Comparison Notes

What stood out to me is that the same overall idea worked on both datasets, but the implementation needed to change depending on the data.

- **Wisconsin:** I could use regular gradient boosting and spend more time on interpretation.
- **Brazil:** I needed a more efficient model setup and more careful evaluation because of the dataset size and class imbalance.

The main points I would highlight from this notebook are:

1. Gradient boosting builds trees one at a time, with each new tree trying to improve the earlier model.
2. Smaller learning rates usually mean the model needs more boosting rounds.
3. Depth, leaf size, and related settings matter because they control how easily the model can overfit.
4. For imbalanced data, accuracy by itself is not enough, so precision-recall metrics matter more.
5. Even when the method stays the same, the workflow should match the dataset you are working with.